Assignment 4 - REST client - C++ 100%

2.4.2024

Poengsum frakoblet modus:
100%

Legg til kommentar

Anonym vurdering: **nei**

26.4.2024

Detaljer

Introduction

- Before you start make sure you have completed Course preparations.
- Each assignment below is a separate directory. When you create each project the directory path will be *ikt103g24v/assignments/solutions/assignment_?_?*.
- **Client library requirement: C++ REST SDK**
- **Note: Use the C++ REST SDK examples link in the lecture to learn how to make other request types**

This is probably the most complex assignment this semester. If you are missing points on other assignments you should probably do those first.

Testing in CLion

For this assignment you need an API server to test against. If you want to test by running the solution in CLion normally you can run my test server with the following command in a separate Terminal/Command Prompt:

```
docker build . -f Dockerfile.assignment_4_server -t ikt103/assignment_4_server && docker run --rm -i
t --init -p 5000:5000 ikt103/assignment_4_server
```

Press Ctrl+C to stop this server when you are done. **Note:** Does not work from Git Bash. Use Command Prompt on Windows.

This is done automatically if you test with *./test.sh 4* so no need to do it manually if you test this way.

4.1 Accessing JSON APIs

Create a program that displays the following menu to the user:

1. Read all students
2. Get student by id
3. Add student

4. Edit student
5. Remove student
6. Exit

then performs the choice entered by the user.

Note: The address of the server is *localhost* and the port is *5000*. This must be used as the base URL for all requests.

Note 2: If something is wrong with a request you send, the server will answer with **400 Bad Request**.

1. Read all students

Connects to the server and send a get request to the */students/* resource.

Print information about every student returned by the server.

Expected output: *id: 1, name: Kathryn Boyd, email: kathrynb@uia.no, year: 2024*

2. Get student by id

Asks the user for a student id then connects to the server and send a get request to the */students/<id>* resource.

Print information about the result returned by the server.

Expected output per status code:

- **200 OK**
  id: 1, name: Kathryn Boyd, email: kathrynb@uia.no, year: 2024
- **404 Not Found**
  Student not found

3. Add student

Asks the user for the following (in order):

1. Name
2. Email
3. Year

Then sends a POST request to */students/* to add the new student. The format of the data must be:

```
{
  "name": "William Canez",
  "email": "williamc@uia.no",
  "year": 2006
}
```

And prints out the response. The server will respond with 201 Created if everything went OK.

Expected output: *Added student: id: 1, name: Kathryn Boyd, email: kathrynb@uia.no, year: 2024*

4. Edit student

This is very similar to add student, so I recommend solving that one first then reusing code.

Asks the user for the following (in order):

1. Student id
2. Name
3. Email
4. Year

Then sends a PUT request to */students/<id>* to modify the existing student. The format of the data must be:

```
{
  "id": 1,
  "name": "William Canez",
  "email": "williamc@uia.no",
  "year": 2006
}
```

Expected output per status code:

- **200 OK** if everything went OK
  Student was edited successfully
- **404 Not Found** if the student doesn't exist
  Student not found

5. Remove student

Asks the user for a student id then sends a DELETE request to /students/<id> to remove the existing student. No data is sent.

Expected output per status code:

- **204 No Content** if everything went OK
  Student was removed successfully
- **404 Not Found** if the student doesn't exist
  Student not found

6. Exit

Exits the program.

Testing and delivery

see *Local testing and Delivery and Bamboo testing* under Assignments.